

Elementos Gráficos

José Gustavo de Souza Paiva

Introdução

- No D3, elementos gráficos são usualmente desenhados em um objeto SVG na página Web
- SVG (*Scalable Vector Graphics*)
  - Conjunto de especificações para desenhos bidimensionais como imagens vetorizadas
  - Criação e manipulação de formas gráficas em documentos HTML
  - Desenhos: documentos XML

Introdução

- A criação de um gráfico/*layout* passa pela criação de uma área de desenho, que é determinada pelas dimensões do SVG (em pixels)

```
<svg width = "500" height = "500">
<!-- definição dos elementos gráficos SVG
</svg>
```

Criação SVG - D3

- Ao criar um SVG, é necessário especificar seu tamanho na tela
- Isso pode ser feito
  - Definindo seus atributos **`width`** e **`height`**, em pixels
    - Tamanho físico real ocupado pelo SVG na página web
  - Definindo seu **`viewBox`**: **`[x1,y1,x2,y2]`**
    - Porção retangular DENTRO do SVG que estará visível

Estrutura básica

```
<html>
 <head>
  <script type = "text/javascript"
   src = "https://d3js.org/d3.v7.min.js">
  </script>
 </head>
 <body>
  <div id = "layout01">
   <svg>
    <!-- elementos -->
   </svg>
  </div>
  <script>
         //Manipulacao objetos
  </script>
 </body>
</html>
```

Observações

- Sistema Cartesiano em SVG
  - Elementos SVG são desenhados em um sistema de coordenadas 2D
  - Coordenadas: por padrão expressas em pixels
  - Objeto possuem atributos de posição no sistema (x e y)
  - Origem do sistema: (0,0) → Canto superior esquerdo
  - Direções dos eixos
    - X: esquerda para direita
    - Y: cima para baixo

Observações

- Ordem de desenho (*z-order*)
  - Elementos são desenhados na ordem em que são declarados
  - Camada “empilhadas”
  - Pode haver sobreposição
  - Ordem de desenho deve ser pensada para obter posicionamento desejado

Exemplo (*z-order*)

```
<!DOCTYPE html>
<html><head>
<script src="https://d3js.org/d3.v7.min.js"></script>
</head><body>
<svg viewbox="0 0 200 100"></svg>
<script>
var svg = d3.select("svg");
svg.append("circle").attr("cx",20).attr("cy",20).attr("r",2)
        .attr("fill","yellow").attr("stroke","black");
svg.append("circle").attr("cx",50).attr("cy",20).attr("r",2)
        .attr("fill","yellow").attr("stroke","black");
svg.append("line").attr("x1",20).attr("y1",20)
        .attr("x2",50).attr("y2",20).attr("stroke","black")
</script></body></html>
```

Formas Básicas

- Retângulo: **`rect`**
- Círculo: **`circle`**
- Elipse: **`ellipse`**
- Linha: **`line`**
- Polilinha: **`polyline`**
- Polígono: **`polygon`**

`Rect`

- Atributos básicos
  - **`x`**: coordenada horizontal topo esquerda
  - **`y`**: coordenada vertical topo esquerda
  - **`width`**: largura
  - **`height`**: altura
  - **`fill`**: cor do preenchimento
  - **`stroke`**: cor do contorno
- Exemplo:

```
<rect x="0" y="0" width="50" height="50" fill="green" />
```

Circle

- Atributos básicos
  - **cx**: coordenada horizontal do centro
  - **cy**: coordenada vertical do centro
  - **r**: raio
  - **fill**: cor do preenchimento
  - **stroke**: cor do contorno
- Exemplo:

```
<circle cx="25" cy="25" r="25" fill="purple" />
```

Ellipse

- Atributos básicos
  - **cx**: coordenada horizontal do centro
  - **cy**: coordenada vertical do centro
  - **rx**: raio horizontal
  - **ry**: raio vertical
  - **fill**: cor do preenchimento
  - **stroke**: cor do contorno
- Exemplo:

```
<ellipse cx="25" cy="25" rx="15" ry="10" fill="red" />
```

Line

- Atributos básicos
  - **x1**: coordenada horizontal inicial
  - **y1**: coordenada vertical inicial
  - **x2**: coordenada horizontal final
  - **y2**: coordenada vertical final
  - **stroke**: cor do contorno
  - **stroke-width**: espessura da linha
- Exemplo:

```
<line x1="5" y1="5" x2="40" y2="40"
stroke="gray" stroke-width="5" />
```

Polyline

- Atributos básicos
  - **points**: lista de pontos 2D (duplas separadas por vírgulas) separados por espaços
  - **stroke**: cor do contorno
  - **stroke-width**: espessura do preenchimento
  - **fill**: cor do preenchimento
- Exemplo:

```
<polyline fill="none" stroke="blue"
stroke-width="2"
points="05,30 15,30 15,20 25,20 25,10 35,10" />
```

Polygon

- Atributos básicos
  - **points**: lista de pontos 2D (duplas separadas por vírgulas) separados por espaços
    - Não é necessário "fechar" o polígono na última coordenada
  - **stroke**: cor do contorno
  - **stroke-width**: espessura do preenchimento
  - **fill**: cor do preenchimento
- Exemplo:

```
<polygon fill="yellow" stroke="blue"
stroke-width="2" points="05,30 15,10 25,30" />
```

Text

- Atributos básicos
  - **x**: posição horizontal baixo-esquerda
  - **y**: posição vertical baixo-esquerda
  - **stroke**: cor da fonte
  - **font-size**: tamanho da fonte
  - **font-family**: tipo da fonte
- Exemplo:

```
<text x="30" y="40" stroke="blue"
font-family="Arial">Teste</text>
```

Grupos de Objetos

- É possível agrupar objetos com a tag **`g`**
- Dessa forma, todos os objetos dentro de um grupo serão tratados como um só
- Exemplo:

```
<g id="g1" fill="red">
<rect id="base" x="25" y="25" width="10" height="50"/>
<rect id="topo" x="15" y="15" width="30" height="11"/>
</g>
```

Transformações

- Aplicável a objetos e grupos
- Atributo **`transform`**
- Permite modificações de
  - Posição
  - Rotação
  - Escala
  - Deformação

Transformações

- **`translate(tx,ty)`**
  - Movimenta o objeto/grupo ao longo do plano de visulização
- Parâmetros básicos
  - **`tx`**: movimentação horizontal
  - **`ty`**: movimentação vertical
- Exemplo:

```
<rect x="20" y="20" width="60" height="60"
transform="translate(30 30)" />
```

Transformações

- **`rotate(a,cx,cy)`**
  - rotaciona o objeto/grupo
- Parâmetros básicos
  - **`a`**: ângulo de rotação (em graus)
  - **`cx`** (opcional): coordenada horizontal do centro de rotação
  - **`cy`** (opcional): coordenada vertical do centro de rotação
  - O centro de rotação padrão é a origem corrente do sistema de coordenadas
- Exemplo:

```
<rect x="20" y="20" width="60" height="60"
transform="rotate(30,50,50)" />
```

Transformações

- **`scale(sx,sy)`**
  - Escala o objeto gráfico
- Parâmetros básicos
  - sx: fator horizontal de escala
  - sy: fator vertical de escala (opcional)
  - Escala é feita em relação à origem corrente do sistema de coordenadas
- Exemplo:

```
<rect x="20" y="20" width="60" height="60"
transform="scale(3)" />
```

Transformações

- Observações
  - Sequências de transformações podem ser aplicada simultaneamente a um objeto
  - Ordem das transformações é importante!

```
<rect x="20" y="20" width="60" height="60"
transform="rotate(30) translate(70,70)">
```

Rotaciona depois translada

```
<rect x="20" y="20" width="60" height="60"
transform="translate(70,70) rotate(30)">
```

translada depois rotaciona

Transformações

- Observações
  - Transformações alteram o sistema de referência
    - Sistemas locais de coordenadas são criados
  - Exemplo:

```
<g transform="translate(100,50)">
   <circle cx="0" cy="0" r="20"/>
</g>
```

  - Grupo foi movido para (100,50)
  - Círculo continua em (0,0) LOCALMENTE
    - Mas na prática, (0,0) → (100,50)

Criação de Formas via JavaScript

- Elementos gráficos podem ser inseridos no *layout* com código Javascript
- Inserção dinamica de acordo com dados e/ou interações do usuário

Criação de Formas via JavaScript

- Processo básico
  - Seleção de nó *container*
  - Criação de *containers* internos (opcional)
  - Adição de nós para elementos gráficos (um para cada)
  - Configuração de atributos e estilos dos elementos
  - Possível aplicação de transformações geométricas
- Dados guiam a construção/configuração dos elementos gráficos

Paths

- Desenhos criados a partir de um caminho percorrido por uma "caneta virtual"
- Qualquer objeto gráfico básico pode ser obtido
- É possível utilizar traços retos ou curvos
  - Linha reta
  - Arcos
  - Curvas de Bézier cúbicas/quadráticas
- Mini-linguagem para criação dos caminhos

Paths

Command	Parameters	Repeatable	Explanation
Pen Command			
M (m)	x, y	Yes	**moveto** Move the pen to a new location. No line is drawn. All path data must begin with a 'moveto' command.
Line Commands			
L (l)	x, y	Yes	**lineto** Draw a line from the current point to the point (x,y).
H (h)	x	Yes	**horizontal lineto** Draw a horizontal line from the current point to x.
V (v)	y	Yes	**vertical lineto** Draw a horizontal line from the current point to y.

Retirado de https://www.dashingd3js.com/svg-paths-and-d3js

Paths

Command	Parameters	Repeatable	Explanation
Cubic Bezier Curve Commands			
C (c)	x1 y1 x2 y2 x y	Yes	**curveto** Draw a cubic Bézier curve from the current point to the point (x,y) using (x1,y1) as the control point at the beginning of the curve and (x2,y2) as the control point at the end of the curve.
S (s)	x2 y2 x y	Yes	**shorthand/smooth curveto** Draw a cubic Bézier curve from the current point to (x,y). The first control point is assumed to be the reflection of the last control point on the previous command relative to the current point. (x2,y2) is the second control point (i.e., the control point at the end of the curve).

Retirado de https://www.dashingd3js.com/svg-paths-and-d3js

Paths

Command	Parameters	Repeatable	Explanation
Quadratic Bezier Curve Commands			
Q (q)	x1 y1 x y	Yes	**quadratic Bézier curveto** Draw a quadratic Bézier curve from the current point to (x,y) using (x1,y1) as the control point.
T (t)	x y	Yes	**Shorthand/smooth quadratic Bézier curveto** Draw a quadratic Bézier curve from the current point to (x,y). The control point is assumed to be the reflection of the control point on the previous command relative to the current point.
Elliptical Arc Curve Command			
A (a)	rx ry x-axis-rotation large-arc-flag sweep-flag x y	Yes	**elliptical arc** Draws an elliptical arc from the current point to (x, y). The size and orientation of the ellipse are defined by two radii (rx, ry) and an x-axis-rotation, which indicate how the ellipse as a whole is rotated relative to the current SVG coordinate system. The center (cx, cy) of the ellipse is calculated automatically to satisfy the constraints imposed by the other parameters. large-arc-flag and sweep-flag contribute to the automatic calculations and help determine how the arc is drawn.

Retirado de https://www.dashingd3js.com/svg-paths-and-d3js

Paths

Command	Parameters	Repeatable	Explanation
End Path Command			
Z (z)	none	No	**closepath** Closes the path. A line is drawn from the last point to the first point drawn.

Retirado de https://www.dashingd3js.com/svg-paths-and-d3js

Paths em D3

- D3 oferece recursos para criação de *paths* utilizando uma lista de pontos, tornando o processo mais simples e dinâmico

- Dados necessários
  - Conjunto de coordenadas
  - Forma de preenchimento do "caminho" entre os pontos
    - Vários tipos de caminho

Paths em D3

- Funcionamento:
  - Criar nó **`path`**
  - Disponibilizar dados (coordenadas) para o desenho
  - Criar variável para definir a forma (função) de desenho
  - Utilizar atributo **`d`** do nó **`path`** para associar a forma de desenho com os dados pela variável

Paths em D3

- Disponibilização dos dados
  - Diversas maneiras
  - Precisam de alguma forma ser mapeáveis para uma lista de pontos 2D
  - Exemplos
    - `var p = [1,2,5,8,3,9]`
    - `var s = [[1,3],[10,2],[100,2],[10,23]]`
    - ```
      var lineData = [ {"x":1,"y":5},
                       {"x":20,"y":20},
                       ...
                     ];
      ```

Paths em D3

- Funções de desenho
  - **`Line`**
  - **`Curve`**
  - **`Line Radial`**
  - **`Area`**
  - **`Radial Area`**
  - **`Arc`**
  - **`Symbol`**
  - **`Links`**
  - Etc
      ```

Line

- Forma mais básica de unir os pontos - por uma reta

- Informações mapeáveis
  - **x**: coordenada horizontal dos pontos
  - **y**: coordenada vertical dos pontos

Line

- Exemplo:

```
var s = [[1,3],[10,12],[100,2],[150,23]];

var linha = d3.line();

d3.select('svg').
        append('path').
        attr('d',linha(s)).
        attr('stroke','blue').
        attr('fill','none');
```

Line

- Exemplo:

```
var s2 = [{"x":1,"y":3},{"x":10,"y":12},
          {"x":100,"y":2},{"x":150,"y":23}];

var linha2 = d3.line()
         .x(function(d) { return d.x; })
         .y(function(d) { return d.y; });

d3.select('svg').
  append('path').attr('d',linha2(s2)).
 attr('stroke','blue').attr('fill','none');
```

Line

- Exemplo - Resultado



`Curve`

- Define formas especiais de interpolar a lista de pontos

- Informações mapeáveis
  - **`x`**: coordenada horizontal dos pontos
  - **`y`**: coordenada vertical dos pontos
  - **`tipo`**: tipo pré-definido de curva

`Curve`

- Tipos pré-definidos

```
curveBasis              curveCatmullRomOpen
curveBasisClosed        curveLinear
curveBasisOpen          curveLinearClosed
curveBundle             curveMonotoneX
curveCardinal           curveMonotoneY
curveCardinalClosed     curveNatural
curveCardinalOpen       curveStep
curveCatmullRom         curveStepAfter
curveCatmullRomClosed   curveStepBefore
```

Curve

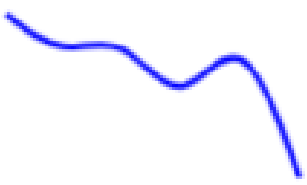
- Exemplo:

```
var cd = [{"x":1,"y":5},{"x":20,"y":20},
          {"x":40,"y":10},{"x":60,"y":40},
           {"x":80,"y":5},{"x":100,"y": 60}
          ];
var curva = d3.line().curve(d3.curveBasis)
                     .x(function(d){return d.x;})
                     .y(function(d){return d.y;});

d3.select("svg").append("path")
.attr("d",curva(cd)).attr("stroke","blue")
.attr("stroke-width",2).attr("fill","none");
```

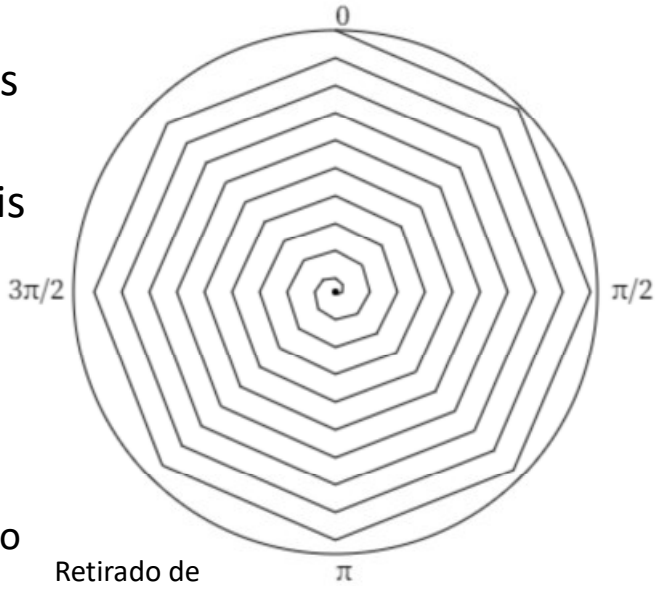
Curve

- Exemplo: Resultado



Line Radial

- Similar à linha, mas utilizando coordenadas radiais
- Informações mapeáveis
  - **`Radius`**: distância do ponto ao centro da circunferência
  - **`Angle`**: valor em radianos (ângulos e raios) para determinação do ponto



Retirado de https://observablehq.com/@d3/d3-lineradial

Line Radial

- Exemplo:

```
var data = [[0, 60],[Math.PI * .25, 60],[Math.PI * .5,
60],[Math.PI * .75, 60],[Math.PI, 60],[Math.PI * 1.25,
60],[Math.PI * 1.5, 60],[Math.PI * 1.75, 60],[Math.PI *
2, 60]];

var radial = d3.lineRadial();
var g = d3.select("svg").append("g");
g.append("path")
  .attr("d",radial(data))
  .attr("stroke","blue").attr("stroke-width",2)
  .attr("fill","none");

g.attr("transform","translate(80,80)");
```

Line Radial

- Exemplo (alternativo):

```
var data = [{a:0,r:60},{a:Math.PI*.25,r:60},
{a:Math.PI*.5,r:60},{a:Math.PI*.75,r:60},{a:Math.PI,r:60},{a:
Math.PI*1.25,r:60},{a:Math.PI*1.5,r:60},{a:Math.PI*1.75,r:60},
{a:Math.PI*2,r:60}];

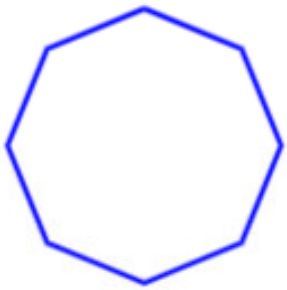
var radial = d3.lineRadial().
             angle(function(d){return d.a;}).
             radius(function(d){return d.r;});

var g = d3.select("svg").append("g");
g.append("path").attr("d",radial(data)).attr("stroke","blue").
attr("stroke-width",2).attr("fill","none");

g.attr("transform","translate(80,80)");
```

Line Radial

- Exemplo: Resultado



Exemplo: https://bl.ocks.org/tlfrd/fd6991b2d1947a3cb9e0bd20053899d6

Area

- Define uma área entre dois limites
  - Por padrão, entre 0 e um conjunto de valores no eixo vertical
- Informações mapeáveis
  - **`x`**: coordenada horizontal
  - **`y0`**: limite inferior da área (valor padrão: 0)
  - **`y1`**: limite superior da área

Area

- Exemplo:

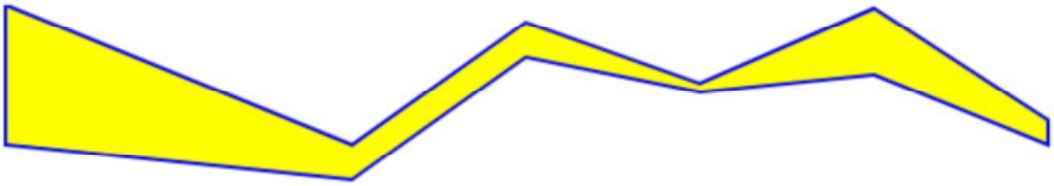
```
var points = [[1,0,80],[200,80,100],[300,10,30],
              [400,45,50],[500,2,40],[600, 66, 80]];

var area = d3.area().
      x(function(d) {return d[0];}).
      y0(function(d) {return d[1];}).
      y1(function(d) {return d[2];});

var svg = d3.select("svg").attr("width","700")
   .attr("height","300")
   .append("path").attr("d",area(points))
   .attr("stroke","blue")
   .attr("stroke-width",2).attr("fill","yellow");
```

Area

- Exemplo: resultado



Arc

- Constrói um arco de raio determinado, começando em um ângulo e terminando em outro
- Atributos mapeáveis
  - **`innerRadius`**: distância do centro ao início da área
  - **`outerRadius`**: distância do centro ao fim da área
  - **`startAngle`**: início da rotação do arco
  - **`endAngle`**: final da rotação do arco
- Atributos de estilo
  - **`padAngle`** e **`padRadius`**: espaçamento entre arcos
  - **`cornerRadius`**: arredondamento das extremidades

Arc

- Exemplo:

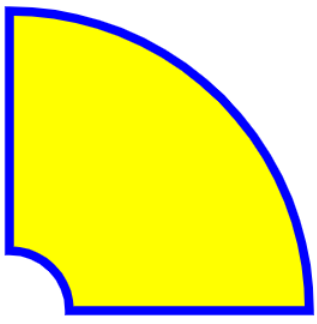
```
var arco = d3.arc().innerRadius(20).outerRadius(100)
      .startAngle(0).endAngle(0.5 * Math.PI);

var svg = d3.select("svg")
   .attr("width","700").attr("height","300")
   .append("g").attr("transform","translate(50,150)");

svg.append("path").attr("d",arco).attr("stroke","blue")
   .attr("stroke-width",3).attr("fill","yellow");
```

Arc

- Exemplo: resultado



Arc

- Exemplo:

```
var arcData = [
 {sAngle:0,eAngle:0.2,c:"yellow"},
 {sAngle:0.2,eAngle:0.6,c:"red"},
 {sAngle:0.6,eAngle:1.4,c:"green"},
 {sAngle:1.4,eAngle:3,c:"blue"},
 {sAngle:3,eAngle: 2* Math.PI,c:"gray"}
];

function arco(s,e) {
  return d3.arc()
      .innerRadius(20).outerRadius(100)
      .startAngle(s).endAngle(e);
};
```

Arc

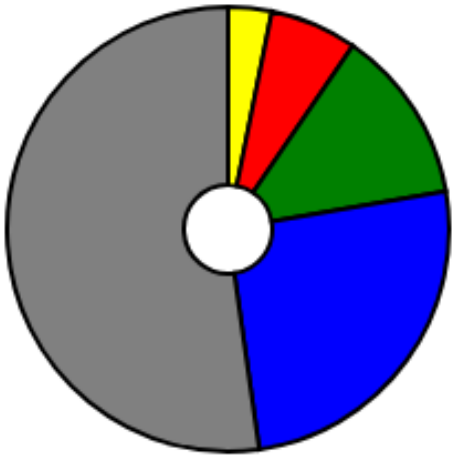
- Exemplo:

```
var svg = d3.select("svg")
.attr("width","700").attr("height","300")
.append("g").attr("transform","translate(150,150)");

for (i=0;i<arcData.length;i++) {
  svg.append("path").attr("d",
  arco(arcData[i].sAngle,arcData[i].eAngle)).
  attr("stroke","black").
  attr("stroke-width",2).
  attr("fill",arcData[i].c);
}
```

Arc

- Exemplo: resultado



Arc

- Exemplo (alternativo):

```
var arcData = [
 {sAngle:0,eAngle:0.2,c:"yellow"},
 {sAngle:0.2,eAngle:0.6,c:"red"},
 {sAngle:0.6,eAngle:1.4,c:"green"},
 {sAngle:1.4,eAngle:3,c:"blue"},
 {sAngle:3,eAngle: 2* Math.PI,c:"gray"}
];

var svg = d3.select("svg")
.attr("width","700").attr("height","300")
.append("g").attr("transform","translate(150,150)")
.call(arco);
```

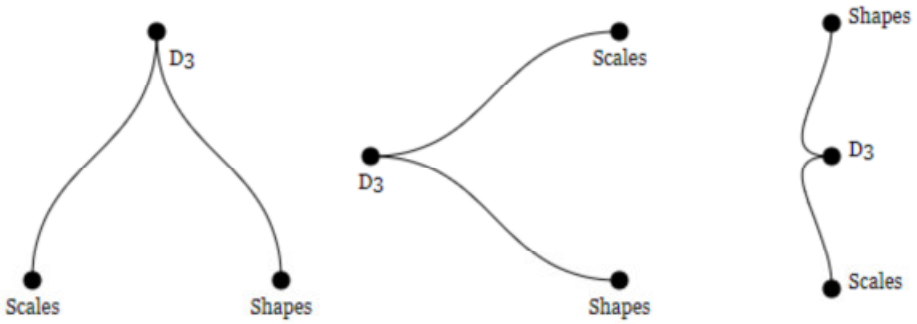
Arc

- Exemplo (alternativo):

```
function arco(selection) {
 for (i=0;i<arcData.length;i++) {
   selection.append("path").attr("d",
    d3.arc()
    .innerRadius(20).outerRadius(100)
    .startAngle(arcData[i].sAngle)
    .endAngle(arcData[i].eAngle))
   .attr("stroke","black")
   .attr("stroke-width",2)
   .attr("fill",arcData[i].c);
 }
};
```

Links

- Criam caminhos entre nós origem e destino
- Variações
  - linkVertical
  - linkHorizontal
  - linkRadial



Retirado de http://using-d3js.com/05_08_links.html

Links

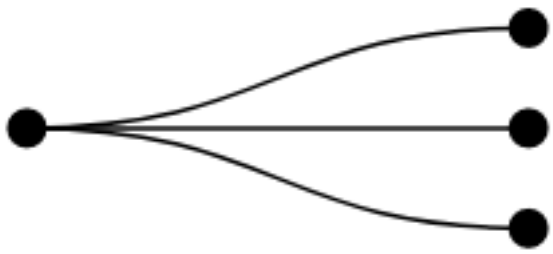
- Exemplo

```
var svg = d3.select('svg').attr('viewBox',[0,0,500,500]);
var nodes = [[[50,50],[175,25]],
             [[50,50],[175,50]],
             [[50,50],[175,75]]];
var origem = svg.append('circle')
       .attr('cx',nodes[0][0][0])
       .attr('cy',nodes[0][0][1]).attr('r',5);

for (i=0;i<nodes.length;i++) {
 var link = d3.linkHorizontal()(
  {source: nodes[i][0], target: nodes[i][1]});
 svg.append('path').attr('d',link)
   .attr('stroke','black').attr('fill','none');
 svg.append('circle')
  .attr('cx',nodes[i][1][0]).attr('cy',nodes[i][1][1])
  .attr('r',5);
}
```

Links

- Exemplo: resultado



Paths em D3 - Outros tipos

- Alguns tipos de paths mais avançados podem ser criados com o uso de associação com dados

- Exemplos
  - https://www.d3indepth.com/shapes/

Referências

- https://www.dashingd3js.com/table-of-contents
- https://www.tutorialsteacher.com/d3js/select-dom-element-using-d3js
- https://bost.ocks.org/mike/selection/